



developers ←

testers ←

→ administrator

→ N/w admin

→ security professional

→ auditors

⋮

DevOps

Responsibilities



Dev Team

- Developers → SRS + DDS
 - Requirement understanding
 - Coding → development
 - Version control → scm / vcs
 - Unit testing
 - Documentation
 - Collaboration → testers + ops team
- Testers
 - Requirement analysis → SRS
 - Test planning, designing and execution → test cases / test suits
 - Defect tracking → bug / issue / gap
 - Reporting bugs to the developers
 - collaboration with devs



Operations Team

- Infrastructure Management → vm / storage / n/w ..
- Security & Compliance → used to create environments
- Deployment & Release Management → deployment in production
- Monitoring, Logging & Alerting
- Incident Management & Troubleshooting
- Cloud & Cost Optimization
- Backup & Disaster Recovery
- Collaboration & Support
- Performance & Capacity Planning



Challenges



Dev Team

- Environment inconsistency
- Delayed feedback loop
- Integration issues
- Manual build and deployment
- Poor visibility after deployment
- Pressure for faster delivery
- Communication gaps



Operations Team

- Frequent and sometimes unstable releases → deployments
- Lack of environment standardization
- Manual configuration management
- Limited collaboration with dev team
- Poor monitoring and alert fatigue
- Downtime and indent pressure
- Security and compliance



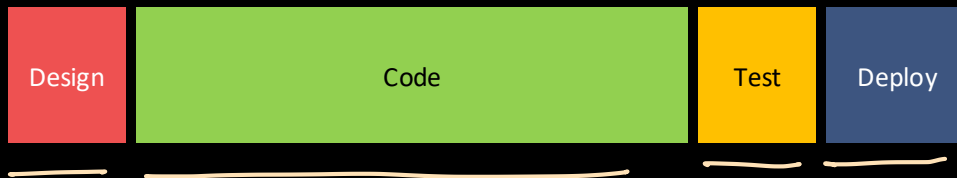
Waterfall Vs Agile



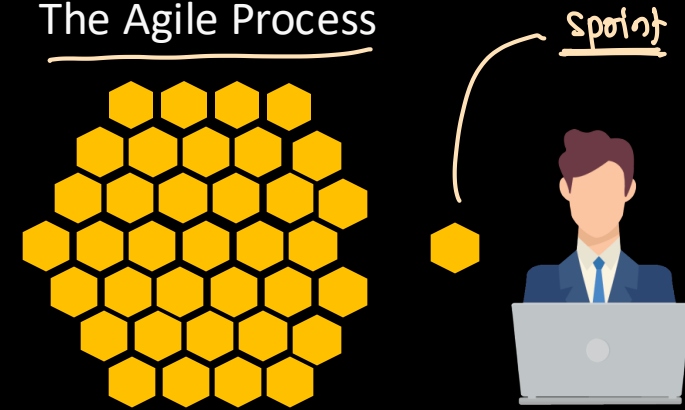
The Waterfall Process



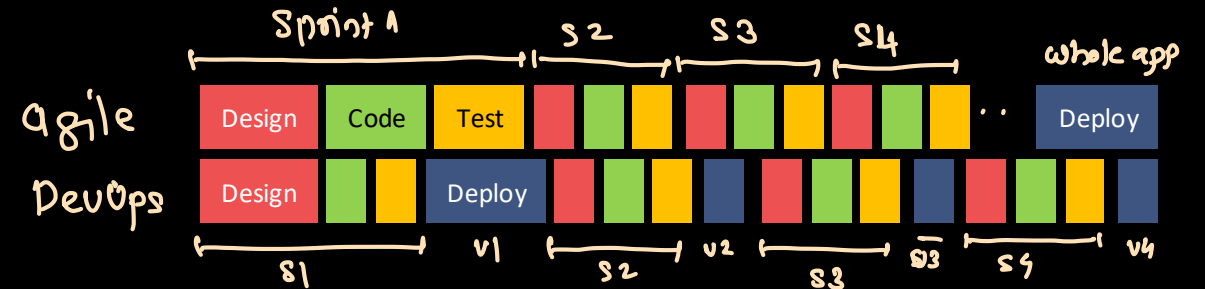
This project has got so big.
I am not sure I will be able to deliver it!



The Agile Process



It is so much better delivering
this project in bite-sized sections

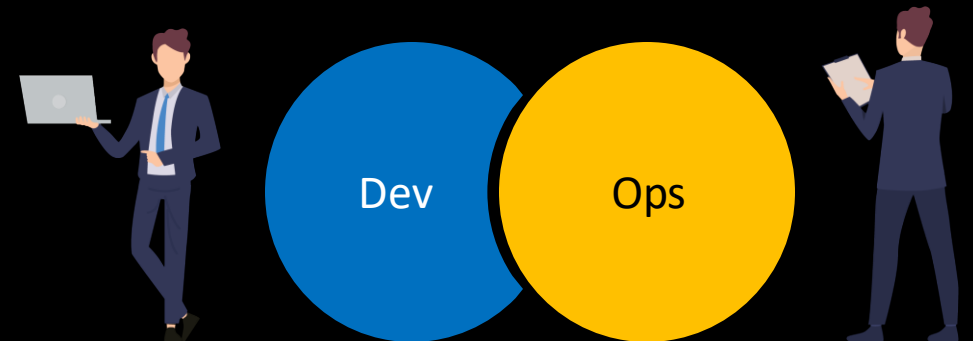


What is DevOps ?

Dev = Developers + Testers



- DevOps is a combination of “Development” and “Operations.”
- It's both a culture and a set of practices/tools aimed at improving collaboration between software developers (Dev) and IT operations (Ops) teams
- The main goal is to deliver software faster, more reliably, and with higher quality — by automating and integrating the processes of software development and IT operations
- It's not just a tool or role, but a culture, methodology, and set of practices that bring together: Developers (who build software) and Operations (who deploy and manage it) to work collaboratively, with the goal of: Delivering software faster, more reliably, and continuously to end-users



How DevOps Solves Challenges



DevOps Solution	What It Fixes	Tools / Practice
Continuous Integration (CI)	Merge & test code frequently	Jenkins, GitHub Actions, GitLab CI
Continuous Delivery (CD)	Automate deployment pipelines	ArgoCD, Spinnaker, Bamboo
Infrastructure as Code (IaC)	Standardize environments	Terraform, Ansible, Puppet
Containerization	Eliminate “works on my machine”	Docker, Kubernetes
Monitoring & Logging	Give both teams visibility	Prometheus, Grafana, ELK Stack
Collaboration Tools	Improve communication	Slack, Microsoft Teams, Jira
Automation Testing	Faster feedback for devs	Selenium, Cypress, JMeter

Goals of DevOps



- **Faster delivery:** Shorten time from idea - deployment
- **Better collaboration:** Break silos between Dev and Ops
- **Automation:** Reduce manual errors and repetitive work
- **Reliability:** Ensure systems are stable and scalable
- **Continuous improvement:** Use feedback to improve processes

Key Principles of DevOps



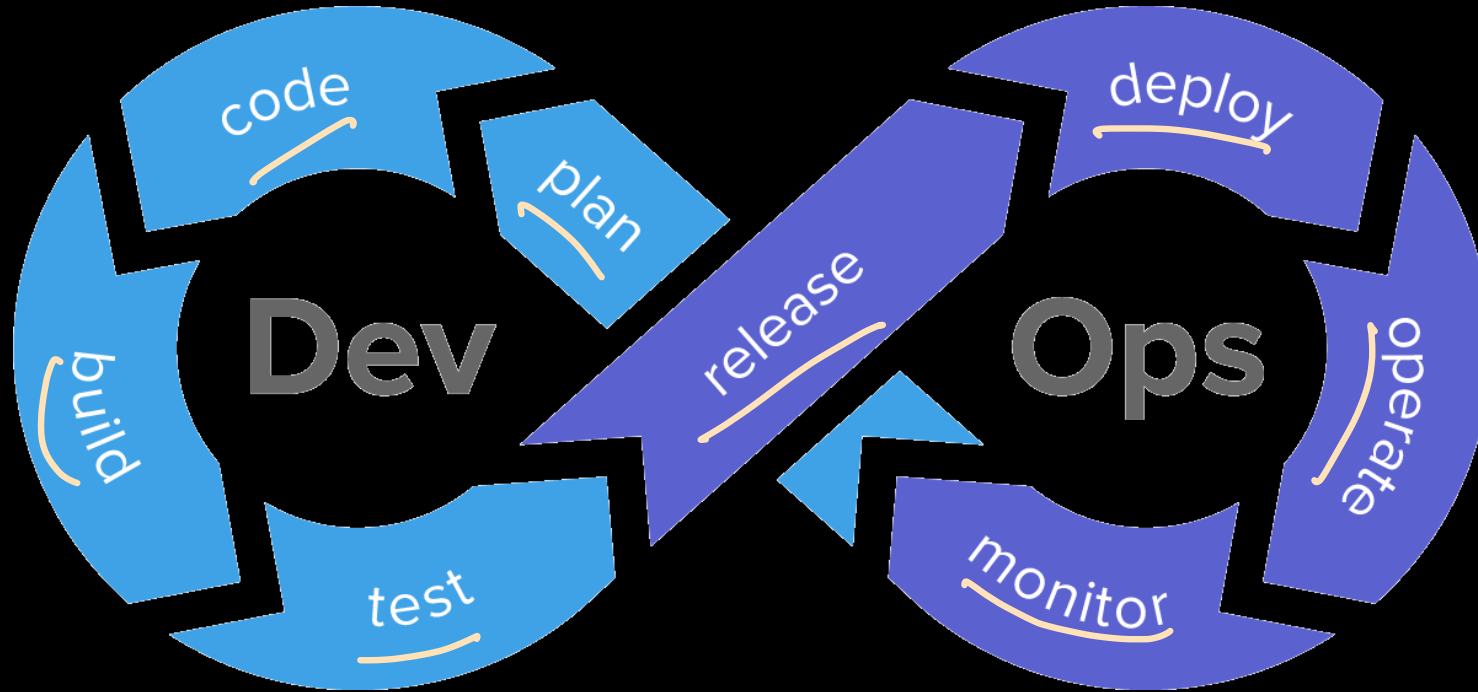
Principle	Description
Collaboration & Communication	Break silos between Dev, QA, and Ops. Everyone shares responsibility for success.
Automation	Automate everything: builds, testing, deployment, monitoring - to reduce manual errors.
Continuous Integration (CI)	Developers merge code frequently and automatically test it.
Continuous Delivery (CD)	Automatically release tested code to staging/production environments.
Continuous Monitoring	Track system health, user feedback, and performance in real-time.
Infrastructure as Code (IaC)	Manage infrastructure (servers, networks) through code, not manual setup.
Continuous Feedback	Learn from real-time data to improve the next iteration.

Reasons to use DevOps



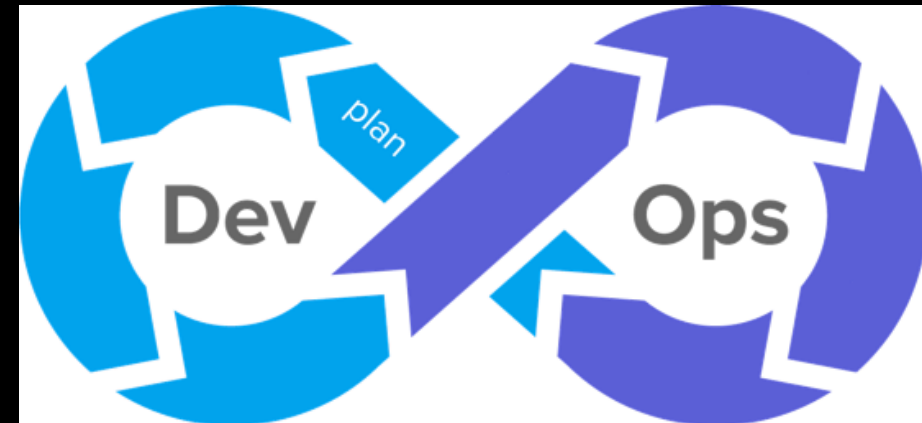
- **Predictability**
 - DevOps offers significantly lower failure rate of new releases
- **Reproducibility**
 - Version everything so that earlier version can be restored anytime
- **Maintainability**
 - Effortless process of recovery in the event of a new release crashing or disabling the current system
- **Time to market**
 - DevOps reduces the time to market up to 50% through streamlined software delivery
 - This is particularly the case for digital and mobile applications
- **Greater Quality**
 - DevOps helps the team to provide improved quality of application development as it incorporates infrastructure issues
- **Reduced Risk**
 - DevOps incorporates security aspects in the software delivery lifecycle. It helps in reduction of defects across the lifecycle
- **Resiliency**
 - The Operational state of the software system is more stable, secure, and changes are auditable

DevOps Lifecycle



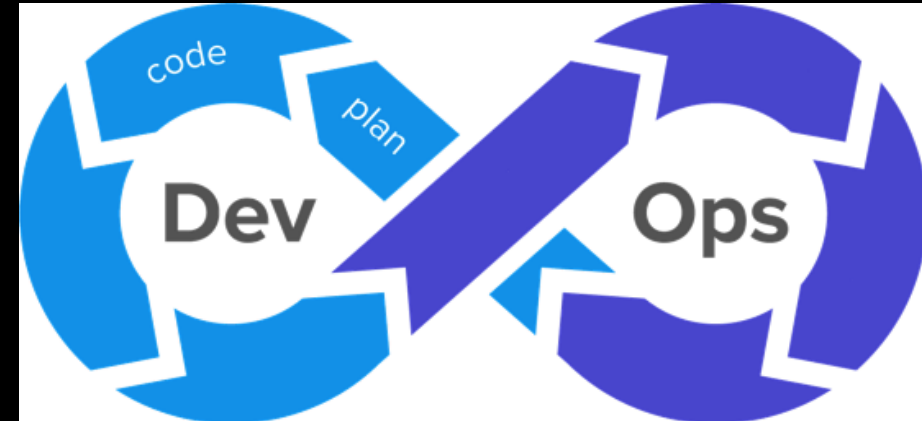
PLAN – Planning and Tracking

- **Goal:** Define project goals, features, and timelines
- **Activities:**
 - Gather and prioritize requirements – SRS / DDS
 - Plan sprints, track progress, and manage tasks
 - Collaborate between development, QA, and operations
- **Common Tools:**
 - Agile Project Management: ¹Jira, ²Trello, Asana, ClickUp, Azure Boards
 - Documentation: Confluence, Notion, Google Docs



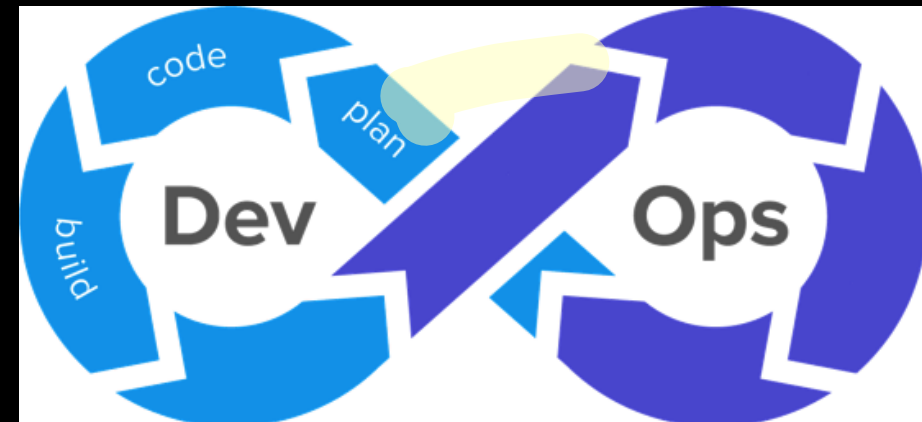
CODE – Development

- **Goal:** Write, review, and manage code efficiently
- **Activities:**
 - Code development using IDEs
 - Version control via Git
 - Code review and collaboration
- **Common Tools:**
 - Version Control: Git, GitHub, GitLab, Bitbucket
 - Code Review: GitHub PRs, Gerrit, Crucible
 - IDEs: VS Code, IntelliJ IDEA, PyCharm, Eclipse
 - Code Quality: SonarQube, Codacy, ESLint, Prettier



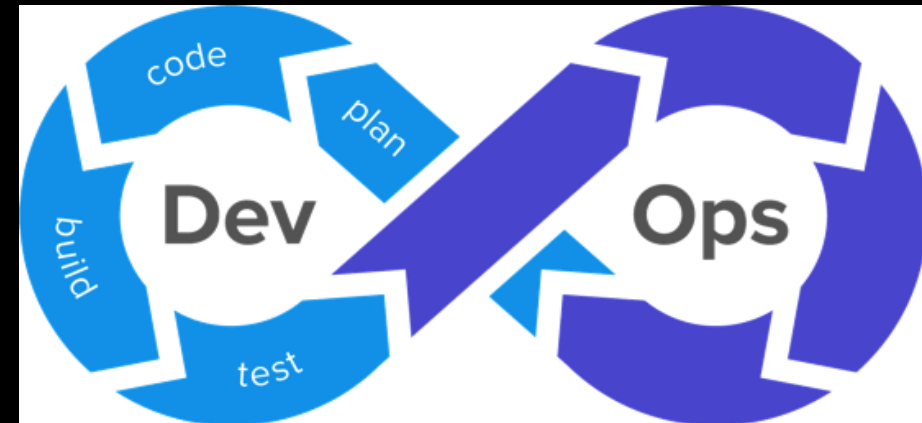
BUILD – Integration and Compilation

- **Goal:** Compile source code into executable artifacts
- **Activities:**
 - Continuous Integration (CI)
 - Build automation and packaging
 - Dependency management
- **Common Tools:**
 - Build Automation: Maven, Gradle, Ant, npm, Make
 - CI Servers: Jenkins, GitLab CI, CircleCI, GitHub Actions, TeamCity
 - Artifact Repository: Nexus, JFrog Artifactory, AWS CodeArtifact



TEST – Automated Testing

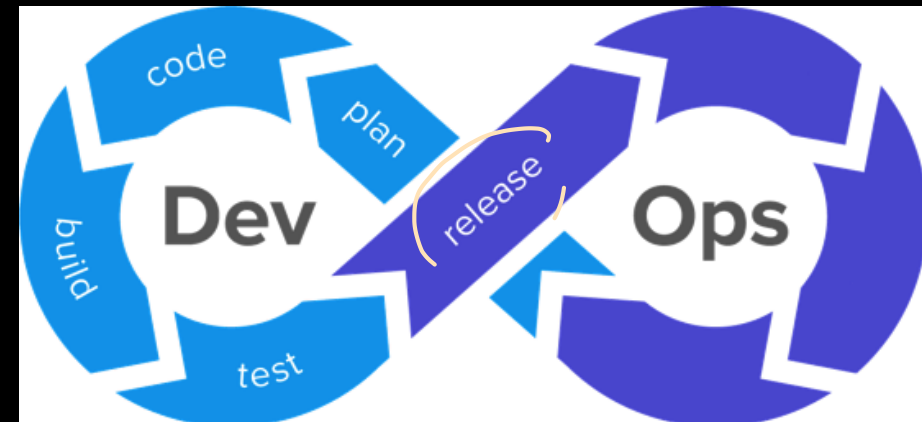
- **Goal:** Ensure software quality and reliability
- **Activities:**
 - Run automated tests (unit, integration, functional)
 - Identify bugs early in the CI pipeline
- **Common Tools:**
 - Unit Testing: JUnit, PyTest, NUnit, Mocha, Jest
 - Functional/UI Testing: Selenium, Cypress, Playwright
 - API Testing: Postman, RestAssured, SoapUI
 - Performance Testing: JMeter, Gatling, k6
 - Security Testing: OWASP ZAP, Burp Suite, SonarQube Security



RELEASE – Versioning and Approval

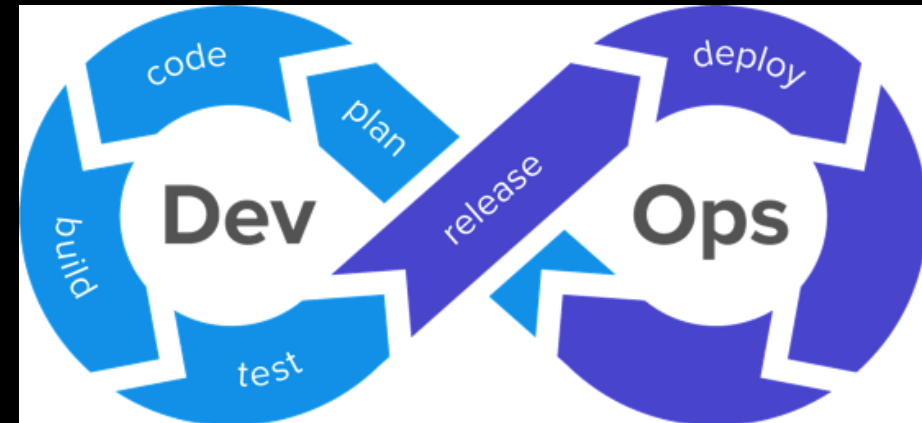


- **Goal:** Prepare the build for deployment into production-like environments
- **Activities:**
 - Version tagging, change approval, and documentation
 - Store tested builds for release
- **Common Tools:**
 - Release Automation: Jenkins, GitLab CI/CD, Bamboo, Azure DevOps, *GitHub Actions*
 - Version Control: Git Tags, Semantic Versioning, GitHub Releases
 - Change Management: ServiceNow, *Jira Service Management*



DEPLOY – Continuous Deployment / Delivery

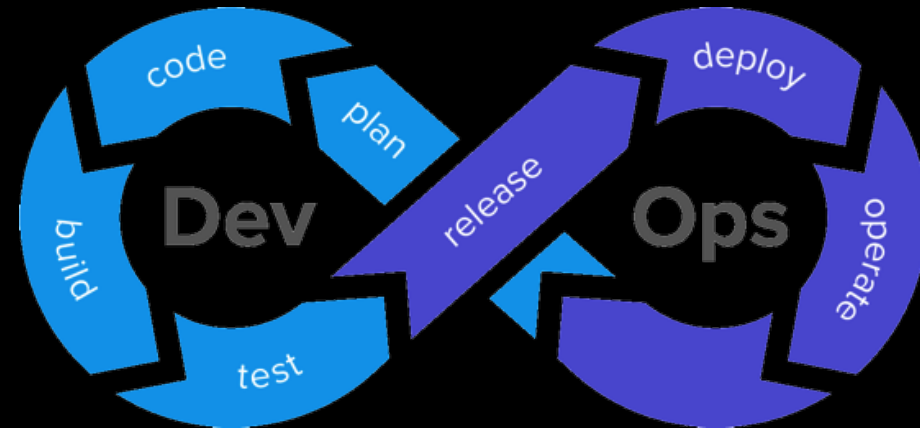
- **Goal:** Deliver the software to target environments (staging, production)
- **Activities:**
 - Continuous Delivery (manual approval)
 - Continuous Deployment (fully automated)
 - Environment configuration via Infrastructure as Code (IaC)
- **Common Tools:**
 - Containerization: Docker, Podman
 - Orchestration: Kubernetes, OpenShift, Docker Swarm
 - IaC (Infrastructure as Code): Terraform, Ansible, Puppet, Chef
 - Cloud Platforms: AWS, Azure, GCP, DigitalOcean



OPERATE – Configuration and Infrastructure Management



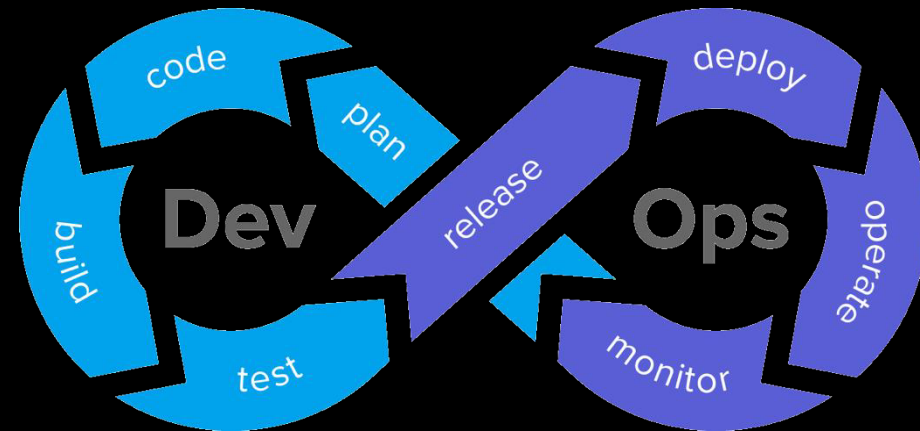
- **Goal:** Manage and maintain systems after deployment
- **Activities:**
 - Monitor uptime, performance, and incidents
 - Automate infrastructure provisioning and scaling
- **Common Tools:**
 - Configuration Management: Ansible, Puppet, Chef, SaltStack, prometheus, Grafana
 - Server Monitoring: Nagios, Datadog, New Relic
 - Log Management: ELK Stack (Elasticsearch, Logstash, Kibana), Fluentd, Splunk



MONITOR – Continuous Feedback



- **Goal:** Continuously observe system health, user experience, and performance
- **Activities:**
 - Collect and analyze metrics
 - Detect failures and trigger alerts
 - Feed insights back into planning phase
- **Common Tools:**
 - Monitoring: Prometheus, Grafana, Zabbix, Datadog
 - Logging: ELK Stack, Graylog, Splunk, Fluentd
 - Alerting: PagerDuty, Opsgenie, Slack Alerts
 - Feedback: Sentry, New Relic, AppDynamics



DevOps Toolchain Example



Stage	Tools
Plan	Jira, Confluence, Notion
Code	Git, GitHub, Bitbucket
Build	Jenkins, Maven, Gradle
Test	Selenium, Cypress, JMeter
Release	GitLab CI/CD, Bamboo
Deploy	Docker, Kubernetes, ArgoCD
Operate	Ansible, Puppet, Terraform
Monitor	Prometheus, Grafana, ELK Stack

Responsibilities of DevOps Engineer

